SUVARNA TECHNOSOFT

POS Machine Integration – Common Wrapper API & Dashboard Overview

1. Introduction

This document explains the POS Machine Integration module developed in our system. Our application supports integration with multiple POS/payment providers through a single common Wrapper API, instead of building and maintaining separate business logic for each vendor across the application.

Integrated POS Vendors:
- Pine Labs
- Razorpay
- Paytm
- Mswipe
- PhonePe
- Worldline

2. Problem Statement

If we directly integrate each POS provider separately in multiple modules/screens:
- Code duplication increases
- Vendor-specific handling becomes difficult
- Maintenance cost increases
- Adding a new POS vendor becomes time-consuming
- Debugging transaction issues becomes complex

Each vendor has different request formats, response structures, authentication methods, transaction status formats, and error messages.

3. Solution Implemented

To solve this, we built:
1. Common Wrapper API – A centralized API layer that handles all POS vendors through a standardized request and response model.
2. POS Dashboard – A monitoring and management dashboard to track transaction status, vendor-wise requests/responses, success/failure counts, retry scenarios, and logs.
3. Centralized Logging – All transaction-related details are stored for audit, troubleshooting, vendor issue analysis, support/debugging, and reconciliation.

4. High-Level Architecture

Flow Overview:
1. Application initiates a payment request
2. Request is sent to Wrapper API
3. Wrapper API identifies target vendor
4. Vendor-specific adapter/service is called
5. Request is transformed into vendor-specific format
6. POS vendor processes transaction
7. Vendor response is normalized into common response format
8. Response is returned back to application
9. Request/Response/Status/Errors are logged
10. Dashboard displays transaction and logs

5. Why Wrapper API?

Benefits of Wrapper API:
- Single entry point for all POS transactions
- Standard request/response contract
- Easier to maintain
- Easy to onboard new vendors
- Reduced UI/business layer complexity
- Better error handling
- Common logging and audit control
- Centralized retry and timeout handling possible

6. Common Wrapper API Responsibilities

The Wrapper API handles vendor identification, request validation, vendor-specific request mapping, API calling / SDK invocation, response transformation, error normalization, transaction status standardization, logging, retry handling (if applicable), timeout handling, and audit trail generation.

7. Standardized Request Model (Concept)

A common request format is used by the application.

Example Common Request Fields:
- Transaction Id
- Bill / Invoice No
- Amount
- Currency
- Payment Mode
- Vendor Name / POS Provider

- Terminal Id / Device Id
- Store / Branch Id
- User Id / Cashier Id
- Patient / Customer Reference (if applicable)
- Callback / Status Tracking Info

8. Standardized Response Model (Concept)

Different vendors return different response payloads. We normalize them into a common structure.

Example Common Response Fields:
- Status (Success / Failed / Pending / Cancelled)
- Transaction Id
- Vendor Transaction Id
- Approval / Auth Code
- Response Message
- Error Code (if any)
- Error Message (if any)
- Card Type / Payment Type (if available)
- Reference Number / RRN (if available)
- Timestamp

9. Vendor-Specific Adapters / Services

Internally, each POS vendor has its own implementation layer.

Example Structure:
- PineLabsService
- RazorpayService
- PaytmService
- MswipeService
- PhonePeService
- WorldlineService

Each service handles authentication/token creation, endpoint selection, vendor request payload build, vendor response parsing, vendor error handling, and status mapping to common enum.

10. Dashboard Features

The POS Dashboard provides operational visibility.

Dashboard can show:
- Total transactions
- Successful transactions
- Failed transactions
- Pending transactions
- Vendor-wise transaction count
- Vendor-wise failure analysis
- Date-wise transaction trends
- Retry / timeout cases
- Request/Response log details
- Error code analysis
- Search by Transaction ID, Invoice/Bill No, Vendor, Date range, Status, Terminal ID

11. Logging Strategy

We are saving all logs for every transaction.

What is logged?
- Request received from application
- Normalized/common request
- Vendor selected
- Vendor request payload
- Vendor API endpoint details (safe metadata only)
- Vendor response payload
- Final normalized response
- Error details
- Exception stack/message
- Processing timestamps
- Retry attempts
- Final transaction status

Why logs are important?
- Debug failures quickly
- Compare vendor issue vs application issue
- Track pending transactions
- Reconcile payment disputes
- Support team troubleshooting
- Audit and compliance reference

12. Key Design Advantages

Business Advantages:
- Faster vendor onboarding
- Better operational control
- Reduced support dependency
- Better reporting visibility

Technical Advantages:
- Loose coupling
- Scalable architecture
- Easier testing
- Better maintainability
- Centralized observability
- Easier future enhancements

13. Real-Time Operational Use Cases

Example 1: New POS Vendor Added
- No major UI changes needed
- Add new vendor service/adapter
- Plug into wrapper API
- Map request/response
- Update dashboard filters if needed

Example 2: Vendor Failure
- Logs help identify root cause
- Error response is standardized
- Support team can see exact request/response trail
- Retry or fallback strategies can be applied later

Example 3: Transaction Pending
- Wrapper stores status
- Dashboard shows pending transactions
- Later reconciliation/status sync can update final status

14. What Juniors Should Understand First

Before working on this module, juniors should clearly understand:
1. POS integration basics
2. Why multiple vendors need abstraction
3. Wrapper API concept
4. Common request/response design

5. Vendor adapter/service pattern
6. Dashboard purpose
7. Logging and audit importance
8. Error handling and status normalization
9. Future scalability considerations

15. Recommended Folder/Code Structure (Suggested)

```
/pos-integration
  /controllers
    PosController
  /services
    PosWrapperService
  /vendors
    PineLabsService
    RazorpayService
    PaytmService
    MswipeService
    PhonePeService
    WorldlineService
  /models
    CommonPosRequest
    CommonPosResponse
    VendorMappings
  /logs
    PosTransactionLogService
  /dashboard
    PosDashboardService
```

16. Important Engineering Notes

Juniors should not assume POS integrations are simple API calls.

Critical areas:
- Timeout handling
- Duplicate transaction prevention
- Idempotency
- Retry safety
- Partial success handling
- Pending reconciliation
- Secure logging (mask sensitive data)
- Token/auth expiry handling
- Vendor downtime scenarios

- Network failures
- Callback/Webhook mismatches (if applicable)

17. Future Enhancements

Possible future improvements:
- Auto retry for safe retryable failures
- Scheduled reconciliation job
- Vendor health monitoring
- Alerting on failure spikes
- Dashboard charts
- SLA monitoring
- Centralized notification for failed transactions
- Fallback vendor routing (if business allows)

18. Summary

We have built a centralized POS Integration platform where:
- Multiple POS vendors are integrated
- A common Wrapper API standardizes processing
- A Dashboard provides visibility
- All logs are stored for audit and troubleshooting
- The system is designed to be scalable, maintainable, and easier to extend

This design reduces complexity in the application and makes multi-vendor POS handling much more efficient.